Architecture SOA et Service Web

Chapitre2:

Validation des documents XML

Réalisé par: Dr. Ghallabi Sameh
l'Institut d'Informatique et Mathématique à Monastir
Cible : 3émé informatique

Validation des documents XML

Comment vérifier qu'un document XML est conforme à une syntaxe donnée?

✓ XML fournit un moyen de vérifier la syntaxe d'un document.

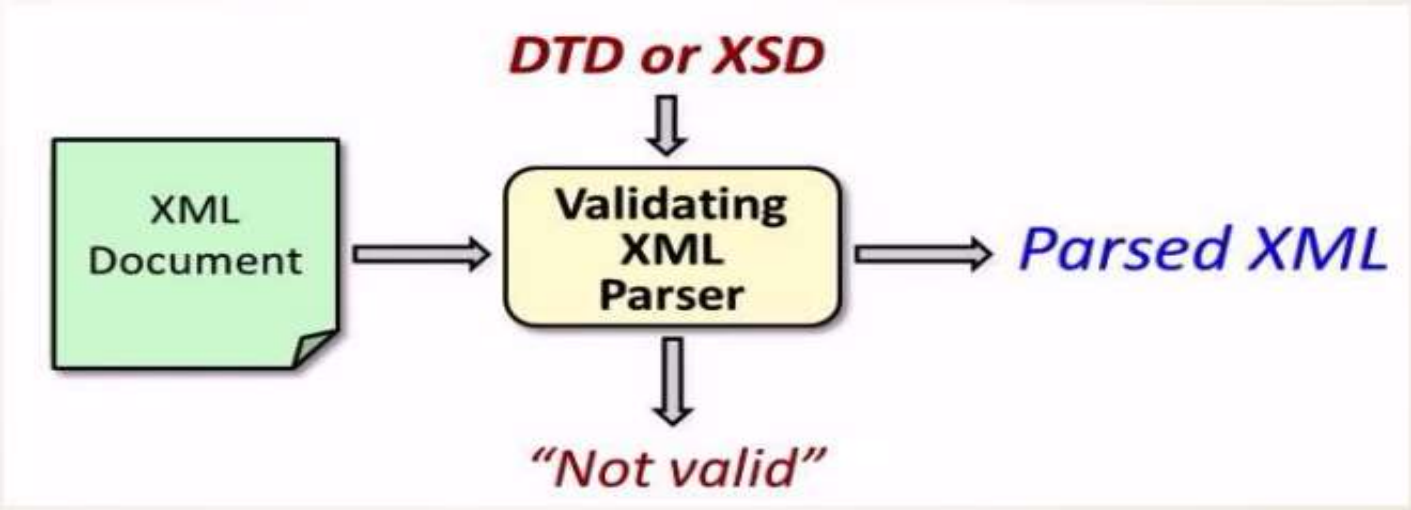
✓Utilisation des XML Schema ou des DTD (Document Type Definition) pour décrire la structure des documents avec les imbrications des éléments possibles.

→Document bien formé : Un document suivant les règles de XML.

→Document valide : Un document XML possédant une DTD (ou XMLSchema) et étant conforme à celle-ci.

Validation des documents XML

Choix entre deux technologies pour écrire les définitions de vos documents XML : les DTD (Document Type Definition) ou les schémas XML.



Validation des documents XML

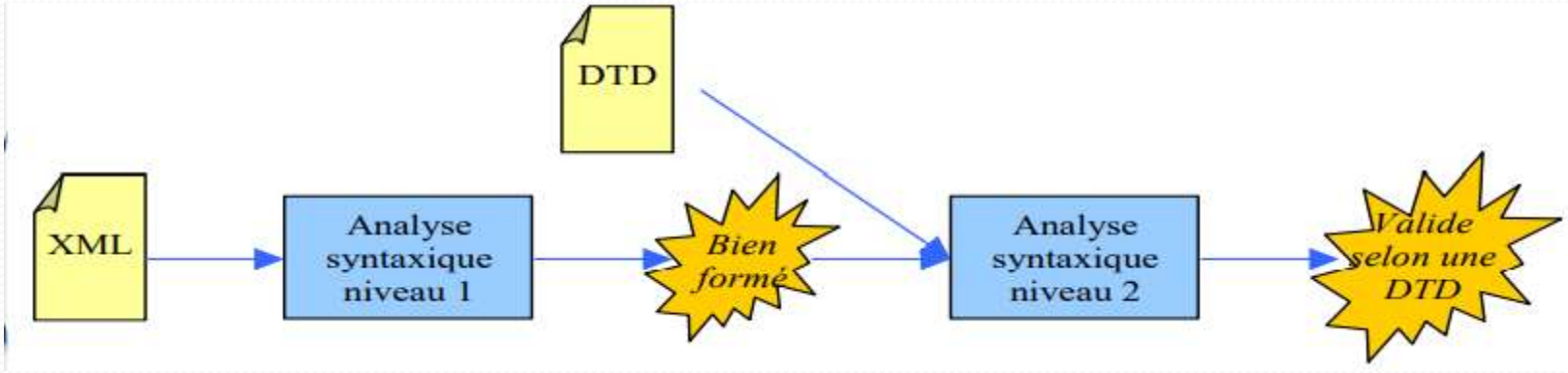
✓Les DTD sont des fichiers spécifiant le vocabulaire et la structure d'un type de documents XML.

✓Une DTD peut être définie de 2 façons :

-Sous forme interne

-Sous forme externe

✓Les règles des DTD internes s'écrivent directement dans le fichier XML alors que les règles des DTD externes sont écrites dans un fichier séparé portant l'extension .dtd .



Validation des documents XML

La déclaration du type de document- DTD

La déclaration d'une DTD doit apparaitre après la déclaration XML, mais avant l'élément racine.

Une DTD interne suit la syntaxe suivante :

```
<!DOCTYPE élément-racine [
déclaration des éléments
]>
```

Validation des documents XML

Création d'une DTD

✓Pour définir les règles portant sur les éléments (balises), on utilise le mot clef ELEMENT.

✓Le mot-clef balise est à remplacer par le nom de la balise à laquelle vous souhaitez appliquer la règle

✓ Dans le cas où la balise contient une valeur simple (du texte), on utilisera le mot clef #PCDATA.

✓Le caractère # est là pour éviter toute ambiguïté avec une balise et indique au parseur qu'il s'agit d'un mot-clé. PCDATA signifie Parsable Character DATA.

✓Une valeur simple désigne par exemple une chaîne de caractères, un entier, un nombre décimal, un caractère, etc.

Validation des documents XML

Déclaration des éléments

Une déclaration d'un élément fournit le nom de l'élément et décrit son contenu.

Le nom d'un élément utilisé dans le document XML doit être identique à celui déclaré dans la DTD.

Un élément peut:

- contenir du texte,
- contenir d'autres éléments (séquence / choix) ,
- contenir un mélange de texte et d'éléments (contenu mixte),
- être vide.
- quelconque

Validation des documents XML

Déclaration des éléments

- **Type de données :**

Type prédéfini	Description
ANY	L'élément peut contenir tout type de données
EMPTY	L'élément ne contient pas de données
#PCDATA	L'élément doit contenir une chaîne de caractère

- **Exemple :**

```
<! ELEMENT Nom_élement  (#PCDATA) >
```

Validation des documents XML

Déclaration des éléments

- **<!ELEMENT Nom (#PCDATA)>**

```
<Nom> Mouad Elouafi </Nom>
```

- **<!ELEMENT Professeur(Nom,Prénom,Adresse)>**

<!ELEMENT Nom (#PCDATA)>
<!ELEMENT Prénom(#PCDATA)>
<!ELEMENT Adresse(#PCDATA)>

```
<Professeur>
  <Nom>  Hajjar</Nom>
  <Prénom> Hind</Prénom>
  <Adresse>  Panoramique</Adresse>
</Professeur>
```

Validation des documents XML

Un exemple de DTD interne

<?xml version="1.0" encoding="UTF-8" standalone="yes"?>	Comme vous définissez un DTD interne, votre fichier est indépendant (standalone).
<!DOCTYPE parent [	Début du DTD interne avec parent comme élément de racine.
<!ELEMENT parent (garcon,fille)>	Cet élément racine soit parent contiendra les sous-éléments garcon et fille.
<!ELEMENT garcon (#PCDATA)>	#PCDATA indique au Parser XML que l'élément garcon contient des données exprimées en chiffres ou en lettres.
<!ELEMENT fille (#PCDATA)>	Idem pour l'élément fille.
]>	Fin du DTD
<parent> <garcon>Mohamed</garcon> <fille>Amani</fille> </parent>	Racine du document XML. Fin du document XML.

Validation des documents XML

-Occurrence des éléments :

Opérateur	Signification	Exemple
+	L'élément doit être présent au minimum une fois	A+
*	L'élément peut être présent plusieurs fois (ou aucune)	A*
?	L'élément peut être optionnellement présent	A?
|	L'élément A **ou** B peuvent être présents (pas les deux)	A|B
,	L'élément A doit être présent et suivi de l'élément B	A,B
()	Les parenthèses permettent de regrouper des éléments afin de leur appliquer les autres opérateurs	(A,B)+

Validation des documents XML

-Occurrence des éléments :

Exemple

```
<!-- choix parmi deux éléments -->
<!ELEMENT mobile (train | avion)>
<!-- séquence d'éléments -->
<!ELEMENT adresse (numéro, voie, ville)>
<!-- Au moins un élément -->
<!ELEMENT carnetDAdresse (carteDeVisite+)>
<!-- séquence d'éléments dont le dernier peut
     apparaître 0 ou plusieurs fois -->
<!ELEMENT carteDeVisite (prenom, nom, organisme,
                         adresse, tel*)>
```

Validation des documents XML

-Déclaration des attributs :

```
<!ATTLIST nom_element nom_attribut type mode>
```

Type prédéfini	Description
#IMPLIED	signifie que l'attribut est optionnel, c'est-à-dire non obligatoire
#REQUIRED	signifie que l'attribut est obligatoire
#FIXED	signifie que l'attribut sera affecté d'une valeur par défaut s'il n'est pas défini.

Exemple:

```
<!ATTLIST homme age CDATA #FIXED "40 ans">
```

Validation des documents XML

-Les types d'un attribut :

CDATA : texte

ID : ne peut pas prendre la même valeur

Énuméré : choix des valeurs

Exemple :

- <!ELEMENT Texte (#PCDATA) >
  <!ATTLIST Texte Langue CDATA #REQUIRED
- <!ELEMENT Photo EMPTY>
  <!ATTLIST Photo format (JPG|PNG) #EMPLIED "PNG"

Validation des documents XML

La déclaration du type de document- DTD externe

✓Elle est introduite avant la première balise (racine) de votre document sous cette forme :

```
<!DOCTYPE racine SYSTEM "URI vers la DTD">
```

- racine est le premier élément (la première balise).
- L'URI peut être absolue ou relative au document.

Exemple:

```
<!DOCTYPE cours SYSTEM "cours.dtd">
```

Dans cet exemple, la DTD cours.dtd est localisée relativement à notre document XML.

DTD externe, interne

Les DTD sont des fichiers spécifiant le vocabulaire et la structure d'un type de documents XML.

Interne : directement en entête du fichier XML.
Externe : en spécifiant l'adresse de la DTD.

standalone="..." (valant "yes" ou "no" suivant si on veut utiliser une DTD externe ou non).
standalone="yes" : pour dire au logiciel que la DTD est définie dans le document.
standalone="no" : pour dire au logiciel d'aller chercher une DTD externe.

Exercices

Soit la DTD « annuaire.dtd »

```
<!DOCTYPE annuaire [
<!ELEMENT annuaire (personne)+>
<!ELEMENT personne   (nom,prenom,email+,tel*)>
<!ATTLIST personne profession (etudiant | professeur | assistant) "etudiant">
<!ELEMENT nom (#PCDATA)>
<!ELEMENT prenom (#PCDATA)>
<!ELEMENT email (#PCDATA)>
<!ELEMENT tel (#PCDATA)>
<!ATTLIST tel type (fixe | portable) #REQUIRED>
]>
```

Ecrivez un fichier XML valide par rapport à cette DTD.

Exercices

```
<?xml version="1.0" encoding="UTF-8"?>
<annuaire>
<personne profession="assistant">
 <nom> ELOUAFI</nom>
<prenom> MOUAD </prenom>
<email> mouad.gsm@gmail.com </email>
<tel type="portable"> 0606681200 </tel>
</personne>
<personne>
 <nom> HAJJAR</nom>
<prenom> HIND</prenom>
<email> hajjar.hind99@gmail.com </email>
<email> H.HAJJAR@gmail.com</email>
</personne>
</annuaire>
```

Limites des DTD

✓DTD permet d'exprimer des contraintes assez basiques
- – Liste des éléments et de leurs attributs
- – Règles de structuration des éléments

Mais:

✓ Pas de nombre précis d'occurrences d'un élément dans une grammaire XML : «*» (0 à N), «?» (0 ou 1), «+» (1 à N)

✓Un seul type des données est défini (du texte)

✓Ne permettent pas de typer des données. Impossible de préciser si une donnée est un nombre entier, un nombre décimal, une date, etc.

→ Autres langages de schéma plus complets, par ex. XML Schema

Schéma XML

- ✓ Les schémas XML permettent, comme les DTD, de définir des modèles de documents.
- ✓ Il est ensuite possible de vérifier qu'un document donné est valide pour un schéma c'est-à-dire respecte les contraintes données par le schéma.

- ✓ Un schéma XML définit :
  - La signification et les relations entre différents éléments et leur contenu.
  - -Les attributs et leurs valeurs
  - -Les types de données, les attributs associés à ces données.

Structure de base d'un schéma XML

- ✓ Comme tout document XML, un schéma XML commence par un prologue et a un élément racine.
- ✓ Le schéma XML utilise un espace de nom xsd.
- ✓ L'élément racine est l'élément xsd:schema

```
<?xml version="1.0" ?>
<xsd:schema …>
  <!-- déclarations d'éléments, d'attributs et de types ici -->
        …
</xsd:schema>
```

Exemple: Le document XML-Schéma ("reference.xsd")

```
<?xml version="1.0" encoding="ISO-8859-1"?>
    <xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
        <!--déclaration d'éléments, d'attributs et de types ici -->
    </xsd:schema>
```

Élément racine

Tout élèment dans un schéma XML doit commencer par le préfixe **xsd (ou xs)**

Déclaration des éléments

- ✓ Dans un schéma, un élément se déclare avec la balise < xsd:element >
- ✓ L'attribut name de xsd:element contient le nom de l'élément.
- ✓ Les éléments composés d'attributs et de sous éléments devront inclure une balise xsd:complexType.

Exemple:

```
<?xml version="1.0" ?>
<xsd:schema …>
    <xsd:element name="remarque" type="xsd:string"/>
    <xsd:element name="contacts" type="typeContacts"/>
    <!-- déclarations de types ici -->
</xsd:schema>
```

- ✓ Ce schéma déclare deux éléments :

  remarque et contacts

- ✓ A chaque élément est associé un type via l'attribut type
  - -remarque de type xsd:string, type simple prédéfini de XML Schema
  - -contacts de type typeContacts, type complexe défini par l'utilisateur

Déclaration des éléments

✓ Un élément peut avoir un type:

1. Type simple: si sa valeur a un type prédéfini en XML-Schéma ou une extension de ces types.
   1.1. Types primitifs: string, boolean, float, double, integer,…

   1.2. Types dérivés : dérivent d'un type primitif ou d'un autre type dérivé.
      1.2.1 Dérivation par restriction,
      1.2.2 Dérivation par extension,
      1.2.3 Dérivation par union,
      1.2.3 Dérivation par liste.

2. Type complexe: s'il contient des sous éléments ou s'il comporte au moins un attribut
      2.1. Collection ordonnée d'éléments typés: xsd:sequence
      2.2. Choix entre éléments typés: xsd:choice
      2.3. Collection non ordonnée d'éléments typés: xsd:all

Types simples : Primitif

- ✓ Dans un schéma XML, un élément simple se déclare avec la balise xsd:element.
- ✓ Un élément simple est un élément qui ne possède pas de sous-éléments.
- ✓ Un type simple, c'est par exemple un chiffre, une date ou encore une chaîne de caractères.

Exemple de déclaration d'un élément de type simple

```
<xsd:element name="Reponse" type="xsd:boolean"> </xsd:element>
```

Quelques types prédéfinis

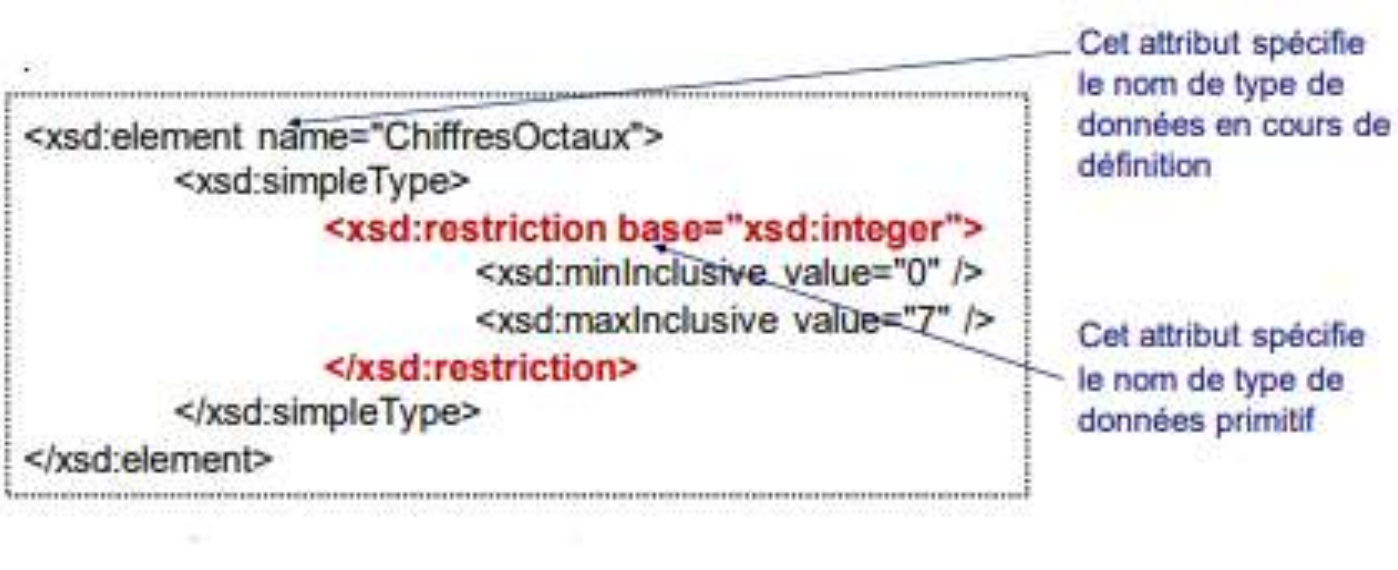
String	Bonjour
Boolean	{true, false, 1, 0}
……	

Dérivation de types simples

Dérivation par restriction (1/3)

- ✓ La dérivation par restriction restreint l'ensemble des valeurs d'un type pré-existant.
- ✓ La restriction est définie par des contraintes de facettes du type de base: valeur min, valeur max ...

Exemple:



Les contraintes de facettes (2/3)

on distingue différents types de contraintes, appelés facettes:

Avec les chaines

- ✓ lenght : la longueur en nombre de caractères.
- ✓ minLenght: la longueur minimale.
- ✓ maxLenght: la longueur maximale.

Pour le nombre et les dates

- ✓ enumeration: un ensemble discret de valeurs.
- ✓ maxInclusive / maxExclusive: borne supérieure
- ✓ minInclusive/ minExclusive : borne inférieure

Les contraintes de facettes (2/3)

Exemple1:

```
<xsd:element name="Mois">
        <xsd:simpleType >
                <xsd:restriction base="xsd:string">
                        <xsd:enumeration value="Janvier"/>
                        <xsd:enumeration value="Février"/>
                        <xsd:enumeration value="Mars"/>
                </xsd:restriction>
        </xsd:simpleType>
</xsd:element>
```

énumération des valeurs possibles

Exemple2:

```
<xsd:element name="Mot">
        <xsd:simpleType >
                <xsd:restriction base="xsd:string">
                        <xsd:minLength value='4'/>
                        <xsd:maxLength value='7'/>
                </xsd:restriction>
        </xsd:simpleType>
</xsd:element>
```

Dérivation par extension

- ✓ Dériver un nouveau type par extension consiste à ajouter à un type existant des sous-éléments ou des attributs.
- ✓ On obtient inévitablement un type complexe.

Exemple 1: cas de l'ajout de l'unité de mesure 'unit' à la valeur donnée 'Mesure' (cas d'une base de type simple

```
<xsd:element name="Mesure">
   <xsd:complexType>
      <xsd:simpleContent>
         <xsd:extension base="xsd:integer">
            <xsd:attribute name="unit" type="xsd:string" use="required"/>
         </xsd:extension>
      </xsd:simpleContent>
   </xsd:complexType>
</xsd:element>
```

Dérivation par liste

- Une liste permet de définir un nouveau type de sorte qu'une valeur du nouveau type est une liste de valeurs du type pré_existant (valeurs séparées par espace).

```
<xsd:element name="DebitPossible">
        <xsd:simpleType>
                <xsd:list itemType="integer"/>
        </xsd:simpleType>
</xsd:element>
```

XML : <DebitPossible> -100 238 8585 0 </DebitPossible>

Types complexes

- Un élément complexe est un élément qui peut contenir d'autres éléments ou bien des attributs.
- Un élément complexe se déclare en utilisant la balise xsd:complexType

```
<xsd:element name="personne">
    <xsd:complexType>
    .....
    </xsd:complexType>
```

- Trois façons de composer des éléments dans un type complexe:

  -sequence,

  - choice,

  -all.

Types complexes: Sequence

- Un type sequence est défini par une suite de sous-éléments qui doivent être présents dans l'ordre donné.
- Le nombre d'occurences de chaque sous-élément est défini par les attributs minOccurs et maxOccurs.

Exemple

```
<xsd:complexType name="Commande">
        <xsd:sequence>
            <xsd:element name="Ad_livraison" type="xsd:string"/>
            <xsd:element name="Ad_facturation" type="xsd:string"/>
            <xsd:element name="texte" type="xsd:string" minOccurs="1" />
            <xsd:element name="items" type="xsd:string" maxOccurs="30" />
        </xsd:sequence>
    </xsd:complexType>
</xsd:element>
```

Types complexes: Choice

- ✓ Un seul des éléments listés doit être présent.
- ✓ Le nombre d'occurences possibles est déterminé par les attributs minOccurs et maxOccurs de l'élément.

Exemple 1

.

```
<xsd:complexType name="Couleur">
    <xsd:choice >
        <xsd:element name="Noire" type="xsd:string" />
        <xsd:element name="Bleue" type="xsd:string"  />
    </xsd:choice>
</xsd:complexType>
```

Types complexes: All

- Le type all est utilisé lorsque l'on veut spécifier que les éléments contenus dans un type complexe peuvent apparaître dans n'importe quel ordre.

Syntaxe

```
<xs:element name =" mon_nom ">
   <xs:complexType>
    <xs:all>
        <! -- liste des éléments -->
    </xs:all>
   <! -- listes des attributs -->
   </xs:complexType >
</xs:element >
```

Exemple

```
<xsd:complexType name="Commande">
      <xsd:all>
            <xsd:element name="Ad_livraison" type=" xsd:string "/>
            <xsd:element name="Ad_facturation" type=" xsd:string "/>
            <xsd:element name="texte" type="xsd:string" minOccurs="0" />
            <xsd:element name="items"  type="Couleur" maxOccurs="1" />
      </xsd:all>
</xsd:complexType>
```

Déclaration des attributs

La définition d'attributs associés à un élément se fait dans un élément qui possède les attributs suivants:

Name : nom de l'attribut

Type : type de l'attribut, par exemple xsd:string, xsd:boolean, .

Use : permet de préciser si l'attribut est obligatoire ou optionnel: required (obligatoire), optional (facultatif)

Value : fixed ou default (valeur par défaut)

```
<xsd:element name="cours">
    <xsd:complexType>
        <xsd:attribute name="at1" type ="xsd:string" use="optional" value = "a " />
        <xsd:attribute name="at2" type ="xsd:boolean" use="required" value ="true"/>
    </xsd:complexType>
</xsd:element>
```